Python basics

Programming Language

Created in 1991 by Guido van Rossum

Cross Platform

Freely Usable Even for Commercial Use

Python

"Python is *easy to use, powerful, and versatile*, making it a great choice for beginners and experts alike."

```
public class Main {
  public static void main(String[] args) {
    System.out.println("Hello world!");
  }
}
```


```
print("Hello world!")
```

- **Image processing**
- **Game development**
- **Data science**

*Big names using **Python***

Python Setup

Some versions of Linux come with Python installed. For example, if you have a Red Hat Package Manager (RPM)-based distribution (such as SUSE, Red Hat, Yellow Dog, Fedora Core, and CentOS), you likely already have Python on your system and don't need to do anything else.

Depending on which version of Linux you use, the version of Python varies and some systems don't include the Interactive DeveLopment Environment (IDLE) application. If you have an older version of Python (2.5.1 or earlier), you might want to install a newer version so that you have access to IDLE.

To see which version of Python 3 you have installed, open a command prompt and run

```
$ python3 --version
```

If you are using Ubuntu, then you can easily install Python with the following commands:

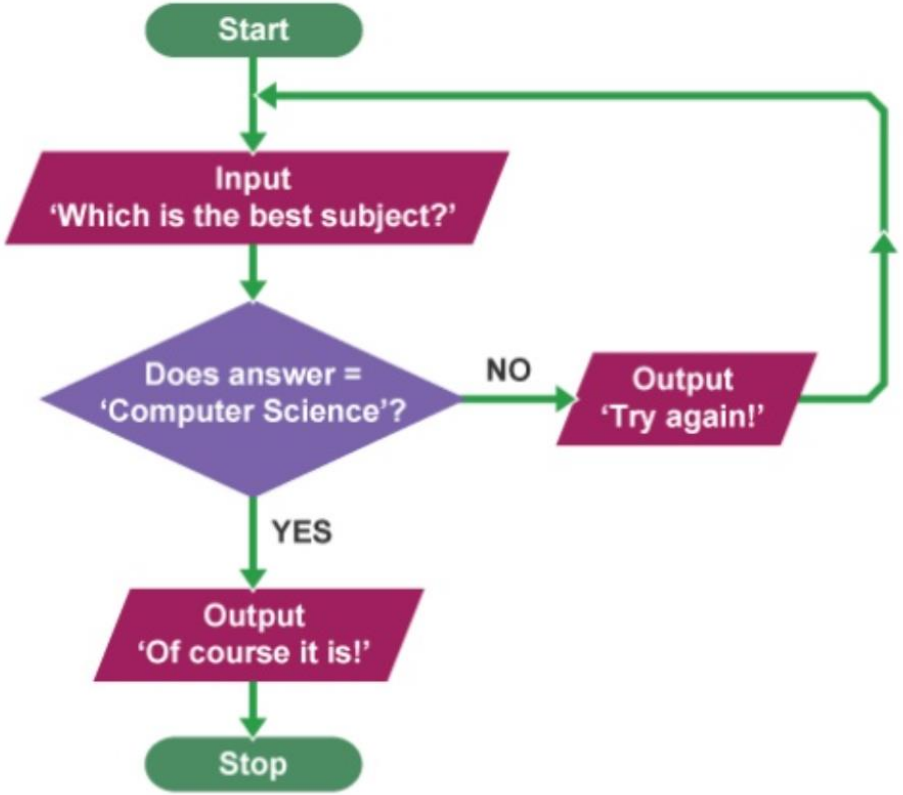
```
$ sudo apt update

$ sudo apt install python3
```

Variables and Data Types

- Variables store and give names to data values
- Data values can be of various types:
  - int : -5, 0, 1000000
  - float : -2.0, 3.14159
  - bool : True, False
  - str : "Hello world!", "K3WL"
  - list : [1, 2, 3, 4], ["Hello", "world!"], [1, 2, "Hello"], [ ]
  - And many more!
- In Python, variables do **not** have types!
- Data values are assigned to variables using "="

```
x = 1 # this is a Python comment
x = x + 5
y = "Python" + " is " + "cool!"
```

Conditionals and Loops



Conditionals and Loops

- Cores of programming!
- Rely on boolean expressions which return either **True** or **False**
  - 1 < 2 : **True**
  - 1.5 >= 2.5 : **False**
  - answer == "Computer Science" : can be **True** or **False** depending on the value of variable answer
- Boolean expressions can be combined with: *and, or, not*
  - 1 < 2 and 3 < 4 : **True**
  - 1.5 >= 2.5 or 2 == 2 : **True**
  - not 1.5 >= 2.5 : **True**

Conditionals: Generic Form

```
if boolean-expression-1:
        code-block-1
elif boolean-expression-2:
        code-block-2
(as many elif's as you want)
else:
        code-block-last
```

Conditionals: Usia SIM (Driving license age)

```
age = 20
if age < 17:
        print("Belum bisa punya SIM!")
else:
        print("OK, sudah bisa punya SIM.")
```

Conditionals: Usia SIM dengan Input

```
age = int(input("Usia: ")) # use input() for Python 3
if age < 17:
        print("Belum bisa punya SIM!")
else:
        print("OK, sudah bisa punya SIM.")
```

Conditionals: Grading

```
grade = int(input("Numeric grade: "))
if grade >= 80:
        print("A")
elif grade >= 65:
        print("B")
elif grade >= 55:
        print("C")
else:
        print("E")
```

Match-case structure (from python3.10)

```
grade = int(input("Numeric grade: "))
match grade:
    case g if g >= 80:
        print("A")
    case g if 65 <= g < 80:
        print("B")
    case g if 55 <= g < 65:
        print("C")
    case _:
        print("E")
```

Ternary Operator in Python

<expression1> if <condition> else <expression2>

```
age = int(input("Age: "))
if age < 18:
        print("Adult")
else:
        print("Minor")
```

```
age = int(input("Age: "))
print("Adult" if age > 17 else "Minor")
```

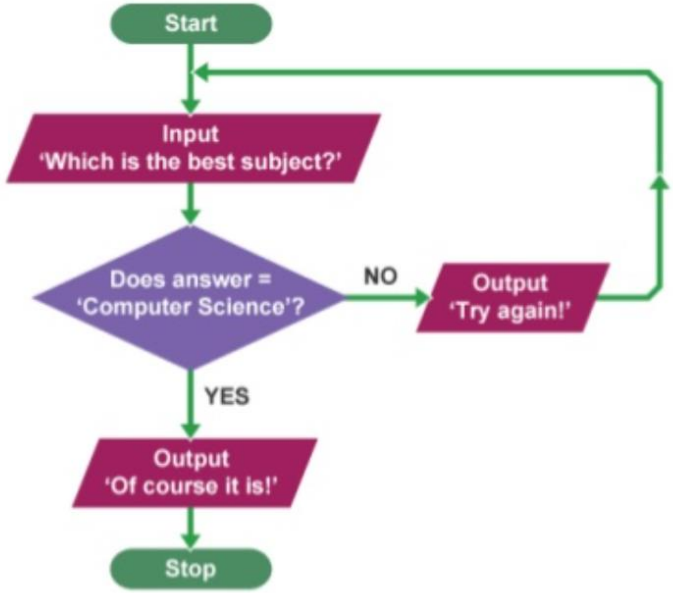
Loops

- Useful for repeating code!
- Two variants:

```
while boolean-expression:
    code-block
```

```
for element in collection:
    code-block
```

While Loops



while boolean-expression: code-block

```
while input("Which is the best subject? ") != "Computer Science":
        print("Try again!")
print("Of course it is!")
```

For Loops

So far, we have seen (briefly) two kinds of collections:

string and list

For loops can be used to *visit each collection's element*:

code-block

for chr in "string": print(chr)

for elem in [1,3,5]: print(elem)

Functions

- encapsulate (= membungkus) code blocks
- Why functions? Modularization and reuse!
- You actually have seen examples of functions:
  - print()
  - input()
- Generic form:

```
def function-name(parameters):
    code-block
    return value
```

Functions: Celcius to Fahrenheit

```
def celsius_to_fahrenheit(celsius):
        fahrenheit = celsius * 1.8 + 32.0
        return fahrenheit
```

def function-name(parameters): code-block return value

Functions: Default and Named Parameters

```
def hello(name_man="Bro", name_woman="Sis"):
        print("Hello, " + name_man + " & " + name_woman + "!")

>>> hello()
Hello, Bro & Sis!
 >>> hello(name_woman="Lady")
Hello, Bro & Lady!
>>> hello(name_woman="Mbakyu",name_man="Mas")
Hello, Mas & Mbakyu!
```

Imports

- Code made by other people shall be reused!
- Two ways of importing modules (= Python files):
  - Generic form: import module_name

    import math

    print(math.sqrt(4))

  - Generic form: from module_name import function_name

    from math import sqrt

    print(sqrt(4))

String

- String is a sequence of characters, like "Python is cool"
- Each character has an index

P	y	t	h	o	n		i	s		c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13

- Accessing a character: string[index]

  x = "Python is cool"

  print(x[10])

- Accessing a substring via slicing: string[start:finish]

  print(x[2:6])

String Operations

P	y	t	h	o	n		i	s		c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13

```
>>> x = "Python is cool"
>>> "cool" in x        # membership
>>> len(x)             # length of string x
>>> x + "?"            # concatenation
>>> x.upper()          # to upper case
>>> x.replace("c", "k") # replace characters in a string
```

String Operations: Split

P	y	t	h	o	n		i	s		c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13

x.split(" ")

P	y	t	h	o	n
0	1	2	3	4	5

i	s
0	1

c	o	o	l
0	1	2	3

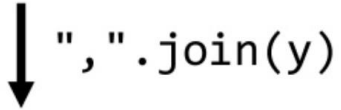
```
>>> x = "Python is cool"
>>> x.split(" ")
```

String Operations: Join

P	y	t	h	o	n
0	1	2	3	4	5

i	s
0	1

c	o	o	l
0	1	2	3



P	y	t	h	o	n	,	i	s	,	c	o	o	l
0	1	2	3	4	5	6	7	8	9	10	11	12	13

```
>>> x = "Python is cool"
>>> y = x.split(" ")
>>> ",".join(y)
```

Input/Output

- Working with data heavily involves reading and writing!
- Data come in two types:
  - Text: Human readable, encoded in ASCII/UTF-8, example: .txt, .csv

U+0041	A
U+0042	B
U+0043	C
U+0044	D
U+0045	E

  - Binary: Machine readable, application-specific encoding, example: .mp3, .mp4, .jpg 42

Input

python

is

cool

cool.txt

```
x = open("cool.txt", "r") # read mode

y = x.read() # read the whole
print(y)

x.close()
```

Input

```
x = open("cool.txt", "r")

# read line by line
for line in x:
    line = line.replace("\n","")
    print(line)

x.close()
```

Input

python
is
cool

cool.txt

```
x = open("C:\\Users\\Fariz\\cool.txt", "r") # absolute location

for line in x:
    line = line.replace("\n","")
    print(line)

x.close()
```

Output

```
# write mode
x = open("carpe-diem.txt", "w")

x.write("carpe\ndiem\n")

x.close()
```

```
# append mode
x = open("carpe-diem.txt", "a")

x.write("carpe\ndiem\n")

x.close()
```

Write mode overwrites files,
while append mode does not overwrite files but instead appends at the end of the files' content

Lists

- If a string is a sequence of characters, **then a list is a sequence of items!**
- List is usually enclosed by square brackets [ ]
- As opposed to strings where the object is fixed (= immutable), we are free to modify lists (that is, lists are mutable).

```
x = [1, 2, 3, 4]
x[0] = 4
x.append(5)
print(x) # [4, 2, 3, 4, 5]
```

List operations

```
>>> x = [ "Python", "is", "cool" ]
>>> x.sort()            # sort elements in x
>>> x[0:2]              # slicing
>>> len(x)              # length of string x
>>> x + ["!"]           # concatenation
>>> x[2] = "hot"        # replace element at index 0 with "hot"
>>> x.remove("Python") # remove the first occurrence of "Python"
>>> x.pop(0) # remove the element at index 0
```

List Comprehension

It is basically a cool way of generating a list

[expression for-clause condition]

Example:

```
[i*2 for i in [0,1,2,3,4] if i%2 == 0]

[i.replace("o", "i") for i in ["Python", "is", "cool"] if len(i) >= 3]
```

Tuples

- Like a list, but you cannot modify it (= immutable)
- Tuple is usually (but not necessarily) enclosed by parentheses ()
- Everything that works with lists, works with tuples, except functions modifying the tuples' content
- Example:

  x = (0,1,2)

  y = 0,1,2 # same as x

  x[0] = 2 # this gives an error

Sets

- As opposed to lists, in sets duplicates are removed and there is no order of elements!
- Set is of the form { e1, e2, e3, ... }
- Operations include: intersection, union, difference.
- Example:

```
x = [0,1,2,0,0,1,2,2]
y = {0,1,2,0,0,1,2,2}
print(x)
print(y)
print(y & {1,2,3}) # intersection
print(y | {1,2,3}) # union
print(y - {1,2,3}) # difference
```

Dictionaries

- Dictionaries map from keys to values!
- Content in dictionaries is not ordered.
- Dictionary is of the form { k1:v1, k2:v2, k3:v3, ... }
- Example:

```
x = {"indonesia":"jakarta", "germany":"berlin","italy":"rome"}
print(x["indonesia"]) # get value from key
x["japan"] = "tokyo" # add a new key-value pair to dictionary
print(x) # {'italy': 'rome', 'indonesia': 'jakarta', 'germany': 'berlin', 'japan': 'tokyo'}
```

Q&A